

Deep-Dive Engineering Guide: EfficientNetV2 Flower Classification

Line-by-Line Code Breakdown, Mathematical Principles, and System Optimization Strategies

Executive Overview & Purpose

This technical guide delivers an exhaustive, line-by-line analysis of the TensorFlow/Kaggle flower classification pipeline across 104 species. Beyond execution, this document elucidates the underlying engineering mechanics, mathematical derivations of custom loss functions, dataset pipeline optimizations, and architecture choices designed to resolve severe class imbalance and prevent VRAM Out-Of-Memory (OOM) failures.

STEP 0: System Configuration & Hardware Optimization

1.1 Mixed Precision Execution Policy

```
from tensorflow.keras import mixed_precision
mixed_precision.set_global_policy('mixed_float16')
```

Code Mechanics: Activates global 16-bit floating-point (FP16) compute operations for heavy tensor calculations (matrix multiplications, 2D convolutions) while retaining 32-bit floating-point (FP32) for master model weights and numerically delicate loss calculations.

Why Mixed Precision Is Essential

VRAM Reduction: Halves memory allocation per tensor, allowing doubling batch sizes or fitting deeper backbones (e.g., EfficientNetV2S) within tight GPU memory limits.

Tensor Core Acceleration: Capitalizes on modern GPU architecture hardware acceleration units designed for low-precision vector math, boosting throughput up to 2–3x.

Numerical Safety: Keras automatically applies dynamic loss scaling under `mixed_float16` to prevent gradient underflow in FP16 representation.

1.2 Multi-Library Seed Pinning for Reproducibility

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
os.environ['PYTHONHASHSEED'] = str(SEED)
```

Code Mechanics: Explicitly synchronizes pseudo-random number generators across Python's built-in random library, NumPy's C-backend RNG, TensorFlow's execution graph kernels, and the operating system's hash seed environment variable.

Why Do It: Deep learning training is inherently stochastic (initial weights, shuffle buffers, dropout masks, augmentations). Fixing seed values ensures identical execution runs across multiple training sessions, enabling

precise attribution of accuracy gains to algorithmic changes rather than random noise.

STEP 1: Exploratory Data Analysis & Tensor Extraction

2.1 TFRecord Binary Parsing & Pixel Normalization

```
def parse_tfrecord(example):
    FEATURE_DESCRIPTION = {
        'image': tf.io.FixedLenFeature([], tf.string),
        'class': tf.io.FixedLenFeature([], tf.int64)
    }
    example = tf.io.parse_single_example(example, FEATURE_DESCRIPTION)
    image = tf.io.decode_jpeg(example['image'], channels=3)
    image = tf.cast(image, tf.float32) / 255.0
    label = tf.cast(example['class'], tf.int32)
    return image, label
```

Code Mechanics:

- `parse_single_example`: De-serializes binary Protocol Buffer records stored inside TFRecord files according to the defined schema dictionary.
- `decode_jpeg(..., channels=3)`: Decompresses raw JPEG byte strings into 3D uint8 image tensors of shape [Height, Width, 3].
- `tf.cast(image, tf.float32) / 255.0`: Converts integer pixel values [0, 255] to floating-point range [0.0, 1.0].

Why Normalize: Standardizes input features to a small symmetric interval, preventing exploding gradients during early backpropagation and maintaining numerical stability across layer activations.

2.2 Extracting Minority Class Tensor for Dynamic Preprocessing

```
class_counts = df_stats['class_id'].value_counts().sort_index().reset_index()
RARE_CLASSES = class_counts[class_counts['count'] < 50]['class_id'].tolist()
RARE_CLASSES_TENSOR = tf.constant(RARE_CLASSES, dtype=tf.int32)
```

Code Mechanics: Calculates sample frequency across all 104 flower species via Pandas, filters out minority classes containing fewer than 50 training images, and converts the Python list into an immutable, compiled TensorFlow constant tensor `RARE_CLASSES_TENSOR`.

Why Convert to Tensor: Pandas DataFrames operate in Python userland memory. Converting minority class IDs into a compiled `tf.constant` enables direct injection into high-performance C++ TensorFlow input pipelines, eliminating Python Global Interpreter Lock (GIL) bottlenecks during mini-batch generation.

STEP 2: High-Performance Data Pipeline & Conditional Augmentation

3.1 Class-Conditional Augmentation Mechanics

```
def conditional_augmentation(image, label):
    is_rare = tf.reduce_any(tf.equal(label, RARE_CLASSES_TENSOR))
    return tf.cond(
        is_rare,
        lambda: data_augment_rare(image, label),
        lambda: data_augment_basic(image, label)
    )
```

Rationale for Conditional Augmentation

Addressing Class Imbalance: Standard pipelines apply uniform augmentation across all samples. However, majority classes already possess sufficient visual variance.

Targeted Rare Augmentation: `tf.cond` dynamically evaluates whether a sample label exists in `RARE_CLASSES_TENSOR`. Rare samples undergo heavy transformations (brightness, contrast, saturation adjustments), synthetically broadening minority feature distribution without overfitting.

Standard Basic Augmentation: Majority samples receive lightweight flips (left-right, up-down), maintaining semantic integrity while saving CPU processing cycles.

3.2 Production-Grade Data Pipeline Architecture

```
def get_training_dataset():
    dataset = tf.data.TFRecordDataset(TRAIN_FILENAMES, num_parallel_reads=AUTO)
    dataset = dataset.map(read_labeled_tfrecord, num_parallel_calls=AUTO)
    dataset = dataset.map(conditional_augmentation, num_parallel_calls=AUTO)
    dataset = dataset.repeat().shuffle(2048).batch(BATCH_SIZE).prefetch(AUTO)
    return dataset
```

Line-by-Line Pipeline Dissection:

- `num_parallel_reads=AUTO`: Concurrently streams data from multiple TFRecord files across disk threads.
- `map(..., num_parallel_calls=AUTO)`: Offloads decoding and augmentation computations across CPU worker threads in parallel.
- `shuffle(2048)`: Maintains a sliding buffer of 2,048 samples in memory to guarantee high-quality uniform random sampling across mini-batches.
- `cache()` (*used in validation*): Caches decoded image tensors in RAM after the first epoch, skipping redundant CPU decoding during subsequent evaluation cycles.
- `prefetch(AUTO)`: Overlaps CPU data preparation with GPU model execution, completely eliminating GPU starvation delays.

STEP 3: Model Architecture & Custom Focal Loss Mechanics

4.1 Pretrained EfficientNetV2S Backbone & Classifier Head

```

base_model = tf.keras.applications.EfficientNetV2S(
    weights='imagenet', include_top=False, input_shape=[224, 224, 3]
)
base_model.trainable = True

inputs = layers.Input(shape=[224, 224, 3])
x = base_model(inputs)
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.BatchNormalization()(x)
outputs = layers.Dense(104, activation='softmax')(x)

```

Architectural Justification:

- **EfficientNetV2S Upgrade:** Replaces legacy EfficientNetB0. Uses Fused-MBConv layers and progressive learning optimizations, delivering superior feature representations with faster training speed.
- **GlobalAveragePooling2D:** Spatial feature map reduction (*Height x Width x Channels* -> *1 x 1 x Channels*). Reduces model parameters dramatically compared to Flatten layers, mitigating spatial overfitting.
- **BatchNormalization & Dropout(0.3):** Stabilizes intermediate layer activation distributions and prevents co-adaptation of features during backpropagation.

4.2 Mathematical Derivation & Code Implementation of Focal Loss

```

class CustomSparseCategoricalFocalLoss(tf.keras.losses.Loss):
    def __init__(self, gamma=2.0, name='custom_sparse_focal_loss'):
        super().__init__(name=name)
        self.gamma = gamma

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.int32)
        epsilon = tf.keras.backend.epsilon()
        y_pred = tf.clip_by_value(y_pred, epsilon, 1.0 - epsilon)
        num_classes = tf.shape(y_pred)[-1]
        y_true_one_hot = tf.one_hot(y_true, depth=num_classes)
        p_t = tf.reduce_sum(y_true_one_hot * y_pred, axis=-1)
        focal_weight = tf.pow(1.0 - p_t, self.gamma)
        cross_entropy = -tf.math.log(p_t)
        loss = focal_weight * cross_entropy
        return tf.reduce_mean(loss)

```

Deep Dive: Focal Loss Mathematical Essence

Mathematical Formulation: Standard Cross Entropy is defined as $CE = -\log(p_t)$. Focal Loss introduces a dynamic modulating factor: $FL(p_t) = -(1 - p_t)^\gamma \log(p_t)$.

Handling Numerical Bounds: `tf.clip_by_value(y_pred, epsilon, 1 - epsilon)` prevents computing $\log(0)$, avoiding disastrous NaN gradient propagation.

True Class Extraction (p_t): `one_hot` expansion combined with `reduce_sum(y_true_one_hot * y_pred)` filters out the specific predicted probability corresponding strictly to the ground-truth target class.

Dynamic Loss Down-Weighting: For easy majority samples where probability confidence is high (e.g., $p_t = 0.9$), the modulating factor with $\gamma=2.0$ becomes $(1 - 0.9)^2 = 0.01$ (a **99% reduction** in loss contribution!). Conversely, for hard rare samples (e.g., $p_t = 0.2$), the weight remains high at $(1 - 0.2)^2 = 0.64$, forcing model gradients to focus almost exclusively on hard, ambiguous, and minority classes.

4.3 Learning Rate Schedule Discrepancy Analysis

```
lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=1e-3, decay_steps=STEPS_PER_EPOCH * EPOCHS
)
# Optimization Note: Compile uses fixed scalar float rather than schedule
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss=focal_loss, metrics=['sparse_categorical_accuracy']
)
```

Critical Engineering Insight: As highlighted in the project comparison notes, while `CosineDecay` is instantiated in code, passing a fixed float scalar `1e-4` into `Adam(learning_rate=1e-4)` overrides the schedule object. To activate dynamic cosine annealing, `lr_schedule` must be passed directly into the optimizer (i.e., `Adam(learning_rate=lr_schedule)`).

STEP 4: Training Callbacks & Memory-Optimized Evaluation

5.1 Model Checkpointing and Early Stopping Callbacks

```
callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        'best_model.h5', save_best_only=True, monitor='val_loss', mode='min'
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=4, restore_best_weights=True
    )
]
```

Code Mechanics: `ModelCheckpoint` continuously monitors validation loss and writes the single best weight snapshot to disk. `EarlyStopping` monitors performance stagnation; if `val_loss` fails to improve for 4 consecutive epochs, training terminates automatically, and `restore_best_weights=True` reverts the model to its optimal parameter state.

5.2 OOM-Safe Batch-Wise Macro F1-Score Computation

```

y_true = []
y_pred = []
for images, labels in val_ds:
    preds = model.predict_on_batch(images)
    y_pred.extend(np.argmax(preds, axis=1))
    y_true.extend(labels.numpy())

final_f1 = f1_score(y_true, y_pred, average='macro')

```

Memory Engineering: Preventing RAM/VRAM Collapses

Legacy Flaw (Old Version): The legacy code executed a custom callback after each epoch that concatenated all validation predictions and true labels into GPU/CPU RAM simultaneously. On large validation sets, this caused fatal Out-Of-Memory (OOM) crashes.

Batch-Wise Resolution (New Version): The refactored code defers metric computation until post-training. By iterating through validation batches using `predict_on_batch`, it processes small localized memory chunks, extracts 1D predicted class indices via `np.argmax`, and accumulates lightweight integer arrays in RAM.

Kaggle Macro F1 Compliance: Unweighted macro F1 calculates precision and recall independently per class before averaging, treating rare classes equal to majority classes.

STEP 5: Inference & Kaggle Submission Packaging

```

# Load optimal weights
model.load_weights('best_model.h5')

# Predict probabilities on test dataset
test_images_ds = test_ds.map(lambda image, idnum: image)
predictions = model.predict(test_images_ds)
probabilities = np.argmax(predictions, axis=-1)

# Extract and decode UTF-8 string test IDs
test_ids_ds = test_ds.map(lambda image, idnum: idnum).unbatch()
test_ids = [idnum.numpy().decode('utf-8') for idnum in test_ids_ds]

# Format DataFrame and save Kaggle submission CSV
sub_df = pd.DataFrame({'id': test_ids, 'label': probabilities})
sub_df.to_csv('submission.csv', index=False)

```

Code Mechanics: Restores optimal model weights, decouples test image tensors from ID tags for batch inference, extracts top-probability class predictions via `argmax`, unbatches byte string ID tensors, decodes them to native UTF-8 strings, and writes a fully compliant `submission.csv` file.

COMPARISON: Comprehensive Comparative Matrix: Old vs. Refactored Pipeline

Architecture Domain	Legacy Version (old.txt)	Refactored Pipeline (new.txt & notes)	Engineering & Impact Rationale
Mixed Precision	Disabled (Standard FP32 computation throughout)	Enabled (mixed_float16 global policy active)	Halves VRAM consumption, speeds up GPU Tensor Core math operations by 2-3x.
Batch Size	BATCH_SIZE = 32	BATCH_SIZE = 16	Reduced batch size compensates for increased VRAM footprint of deeper model backbone.
Model Backbone	EfficientNetB0 (1280 feature maps)	EfficientNetV2S (Fused-MBConv architecture)	Upgraded backbone captures deeper, fine-grained visual representations across 104 species.
Validation Count	Implicit / Uncounted validation size	Explicit Regex count via count_data_items	Allows precise determination of VAL_STEPS in model.fit, preventing epoch truncation.
Macro F1 Evaluation	Per-epoch callback with total RAM concatenation	Post-training batch-wise predict_on_batch	Eliminated recurring OOM memory crashes during validation evaluation cycles.

Summary of Optimization Gains

By combining Mixed Precision FP16, dynamic tf.data pipelines, conditional rare-class augmentation, custom Focal Loss down-weighting, and batch-wise memory evaluation, the refactored pipeline resolves class imbalance and hardware resource limits simultaneously, achieving Kaggle-ready performance and operational stability.